



P R E S E N T S

VYNEDAM TALENT HUNT 2K26

VYNEDAM PRESENTS THE ULTIMATE 36 HOUR CHALLENGE

COMPETE. INNOVATE. WIN



REAL WORLD
PROBLEM SOLVING



CODING &
DEPLOYMENT



36 HOUR NON
STOP HACKATHON



SHOWCASE
& PITCHING

JULY 4 – 5 · 2026 36 HOURS OFFLINE HACKATHON

EXCITING REWARDS



1ST PRIZE
₹50,000



2ND PRIZE
₹30,000



3RD PRIZE
₹20,000

OFFICIAL PROBLEM STATEMENT BOOK · 15 INDUSTRY CHALLENGES

Don't miss your chance to shine — form your team and get ready for 36 hours of innovation.

ABOUT THE EVENT

From idea to a deployed product in 36 hours

VYNEDAM Talent Hunt 2K26 is a 36-hour offline hackathon where teams choose a real industry problem, design a production-oriented solution, and ship a working prototype. Every challenge in this book is written the way a real company scopes engineering work — **a clear problem, market context, requirements, and a reference architecture** you can actually build against.

Real-World Problem Solving

Challenges drawn from live industry pain points across **eight-plus sectors**

Coding & Deployment

You are judged on a **working, deployed build** — not slides alone.

36 Hours, Non-Stop

One problem, one solution, one focused build window.

Showcase & Pitch

Final demo and pitch to a **panel of judges and mentors**

EXCITING REWARDS

1st Prize

₹50,000

2nd Prize

₹30,000

3rd Prize

₹20,000

Internships · Paid internships for top performers · Networking with developers & innovators · Registration Fee ₹350

How to use this book. Each of the 15 problem statements is a self-contained brief. Read the **summary** and **the problem** to understand the why, then use the **requirements** and **architecture** to scope your build. The **evaluation criteria** tell you what judges reward; the **bonus features** are how you stand out if you finish early.

SECTORS COVERED

AgriTech

GovTech

HealthTech

Industry 4.0

ClimateTech

FinTech

Web3

RegTech

Enterprise AI

Manufacturing

EdTech

01 AGRITECH AI Smart Agriculture Platform

DIFFICULTY

STARTUP

A precision-agriculture intelligence platform that fuses satellite imagery, ground IoT sensors and hyperlocal weather to help farmers make data-driven decisions on irrigation, crop health and yield.

CONTEXT

THE PROBLEM

Indian farming is dominated by ~120 million smallholders who irrigate on intuition, detect disease only after visible damage, and lose 15–25% of yield to poor timing. Agronomic advice is generic, delayed and rarely reaches the field.

EXISTING SOLUTIONS & GAPS

Existing tools give satellite indices without last-mile advice, ignore soil-level sensor data, and are priced for large estates, not the 1–5 acre reality of most Indian farms.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Farmer dashboard (web + PWA)** with plot-level NDVI/moisture maps
- **AI irrigation scheduler** with per-crop water-budget recommendations
- **Disease/pest detection from leaf images** with treatment guidance
- **Weather-linked alerts (SMS/WhatsApp)** in regional languages

NON-FUNCTIONAL REQUIREMENTS

- **Offline-first PWA** for low-connectivity villages
- **P95 API latency** < 400ms; alert delivery < 2 min

ARCHITECTURE

SYSTEM ARCHITECTURE

Flutter/React PWA → API Gateway → AI Services (vision + irrigation) → Domain Microservices (plots, alerts, advisory) → PostgreSQL + PostGIS + Vector DB → Analytics Dashboard. Ingestion workers pull satellite tiles and IoT streams via a message queue.

AI ARCHITECTURE

CNN (EfficientNet/ViT) for disease classification; gradient-boosted model for irrigation-need prediction on sensor+weather features; RAG over an agronomy knowledge base for advisory text generation.

DATABASE DESIGN

PostgreSQL for transactional data, PostGIS for geospatial plots, TimescaleDB/Timeseries for sensor streams, Vector DB for advisory retrieval.

RECOMMENDED TECH STACK

Frontend: React/Flutter PWA. **Backend:** Node.js/FastAPI microservices. **AI:** PyTorch, ONNX runtime. **Data:** PostgreSQL+PostGIS, TimescaleDB, Qdrant. **Infra:** Docker, Kubernetes, Kafka.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Accuracy of disease** detection on a held-out leaf dataset
- **Quality and clarity** of irrigation recommendations
- **Last-mile UX** for low-literacy farmers
- **System resilience** under low connectivity

DEPLOYMENT GUIDE

Blue-green deploy on Kubernetes with a staging region; canary the AI models; automated rollback on error-rate spike; IoT firmware OTA channel.

REFERENCES

Sentinel Hub API, ICAR agronomy guidelines, FAO crop water requirement tables.

MARKET SIZE

TAM: \$24B global precision-agriculture market. **SAM: \$4.5B** India AgriTech. **SOM: \$180M** reachable smallholder-advisory segment in 3 years.

FUNCTIONAL REQUIREMENTS

- **Ingest Sentinel-2 imagery** and compute NDVI/NDWI per plot
- **Stream soil-moisture, temperature** and humidity from IoT nodes
- **Image-classification model** for crop-disease identification
- **Rule + ML** hybrid irrigation recommendation engine
- **Multilingual advisory** delivery over SMS/WhatsApp
- **Handle 50k concurrent** plots per region node
- **Data retained with** farmer consent; export on request

CLOUD ARCHITECTURE

Containerised services on Kubernetes; object storage for imagery tiles; managed queue (Kafka/PubSub) for IoT ingestion; edge caching for map tiles; autoscaling per region.

API FLOW

Sensor → Ingestion API → Queue → Processor → DB → Dashboard poll/websocket. Farmer image upload → Vision API → classification → advisory service → notification service.

BONUS FEATURES

- **Yield-prediction model** with confidence bands
- **Marketplace linking farmers** to buyers / mandis
- **Carbon-credit estimation** from sustainable practices

FUTURE SCOPE

Drone-based scouting integration, credit scoring for farm loans, and a national agronomy data cooperative.

An AI assistant for government offices that automates citizen query handling, drafts official documents, routes workflows and provides multilingual support across departments.

CONTEXT

THE PROBLEM

Citizens wait weeks for routine services; officers spend the majority of their day on repetitive drafting and file routing. Language barriers and opaque processes erode trust in public service delivery.

EXISTING SOLUTIONS & GAPS

Existing e-gov portals are form-heavy and static. There is no conversational layer that understands intent, drafts compliant documents, or routes cases with an auditable trail.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Citizen-facing multilingual chat** and voice assistant
- **Officer copilot for** document drafting and summarisation
- **Workflow routing engine** with SLA tracking
- **Full audit log** and role-based access console

NON-FUNCTIONAL REQUIREMENTS

- **Immutable audit trail** for every AI action
- **Data residency** within national boundaries

MARKET SIZE

TAM: \$12B global GovTech automation. **SAM: \$2.1B** India e-governance services. **SOM: \$95M** department-copilot deployments in 3 years.

FUNCTIONAL REQUIREMENTS

- **Intent detection and** multilingual query answering
- **Template-aware document** generation with placeholders
- **Case routing based** on department rules
- **Escalation and** SLA-breach notifications

- **99.9% uptime** for citizen services
- **Explainable outputs** for accountability

ARCHITECTURE

SYSTEM ARCHITECTURE

Web/voice frontend → API Gateway → AI Services (LLM copilot, translation) → Domain Microservices (cases, documents, routing) → PostgreSQL + Vector DB → Analytics Dashboard. A policy engine gates every AI action.

AI ARCHITECTURE

RAG over statutes, circulars and templates; fine-tuned instruction model for drafting; translation models for Indian languages; guardrails and citation enforcement.

DATABASE DESIGN

PostgreSQL for cases/users, Vector DB for policy retrieval, append-only audit store (e.g., WORM bucket or ledger table).

CLOUD ARCHITECTURE

Sovereign/on-prem-capable Kubernetes; encrypted object storage for documents; isolated tenancy per department; centralised audit sink.

API FLOW

Citizen query → Gateway → Copilot → RAG retrieval → response with citations → audit log. Officer draft → template service → LLM fill → human approval → issue.

RECOMMENDED TECH STACK

Frontend: React + Web Speech. **Backend:** FastAPI/Node microservices. **AI:** open LLM + RAG (LangChain/LlamaIndex). **Data:** PostgreSQL, Qdrant/pgvector. **Infra:** Kubernetes.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Correctness and compliance** of drafted documents
- **Multilingual accuracy across** at least 3 languages
- **Completeness of** the audit trail
- **Officer time saved** per task (measured)

DEPLOYMENT GUIDE

Staged rollout per department; feature flags; human-in-the-loop mandatory before any citizen-facing issuance; rollback playbook.

REFERENCES

DigiLocker APIs, national language translation datasets, RTI/records-retention norms.

BONUS FEATURES

- **Grievance-prediction analytics** for proactive service
- **Digital signature** and DigiLocker integration
- **Voice-first kiosk mode** for walk-in citizens

FUTURE SCOPE

Cross-department single-window services and predictive resource planning.

03 INDUSTRY 4.0 TOP PICK

Factory Intelligence OS

DIFFICULTY

STARTUP

An industrial AI operating system that unifies computer vision quality control, predictive maintenance and a real-time digital twin of the shop floor.

CONTEXT

THE PROBLEM

Unplanned downtime costs manufacturers 5–20% of productive capacity. Defects are caught late, maintenance is reactive, and machine data lives in disconnected silos with no unified view.

EXISTING SOLUTIONS & GAPS

Point solutions exist for vision or maintenance separately, but no affordable, unified OS ties sensors, cameras and a live digital twin into one operations picture for mid-sized plants.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Real-time digital-twin dashboard** of lines and machines
- **Vision-based defect detection** on the production line
- **Predictive-maintenance engine** with failure-risk scores
- **OEE analytics** and alerting console

NON-FUNCTIONAL REQUIREMENTS

- **Sub-second telemetry** ingestion at scale
- **Edge inference** for low-latency vision

ARCHITECTURE

SYSTEM ARCHITECTURE

Operator UI → API Gateway → AI Services (vision, RUL, anomaly) → Domain Microservices (assets, twin, OEE) → PostgreSQL + Timeseries + Vector DB → Analytics Dashboard. Edge agents run inference near the line.

AI ARCHITECTURE

Object-detection model (YOLO/Detectron) for defects; LSTM/transformer time-series models for anomaly and RUL; unsupervised clustering for novel failure modes.

DATABASE DESIGN

Timeseries DB for telemetry, PostgreSQL for asset registry, object storage for image evidence, Vector DB for similar-defect search.

RECOMMENDED TECH STACK

Frontend: React + WebGL twin. **Backend:** Go/FastAPI. **AI:** PyTorch, ONNX, edge runtime. **Data:** TimescaleDB, PostgreSQL, MinIO. **Infra:** K3s edge, Kafka/MQTT.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Defect-detection precision/recall** on line footage
- **Lead time of** maintenance predictions vs. actual failures
- **Fidelity and responsiveness** of the digital twin
- **Edge inference** latency

DEPLOYMENT GUIDE

GitOps to edge clusters; staged model promotion via registry; shadow-mode evaluation before switching a model live.

REFERENCES

OPC-UA spec, ISO 22400 (OEE/KPIs), NVIDIA Metropolis references.

MARKET SIZE

TAM: \$16B industrial AI. **SAM: \$3.2B** India smart-manufacturing. **SOM: \$140M** mid-market factory deployments in 3 years.

FUNCTIONAL REQUIREMENTS

- **Ingest PLC/sensor telemetry** and camera streams
- **Anomaly detection** on vibration/temperature signals
- **Defect classification** with bounding boxes
- **Remaining-useful-life estimation** per asset
- **Deterministic alerting with** no missed critical events
- **Horizontal scale** across multiple plants

CLOUD ARCHITECTURE

Hybrid edge-cloud: edge Kubernetes at the plant, cloud for aggregation and model training; MQTT/OPC-UA gateways; model registry with versioning.

API FLOW

Sensor/camera → Edge agent → inference → event bus → twin state update → dashboard + alert. Failure event → maintenance workflow.

BONUS FEATURES

- **AR overlay for** technicians via tablet
- **Energy-optimisation recommendations**
- **Automated root-cause** analysis reports

FUTURE SCOPE

Autonomous line rebalancing and supplier-quality feedback loops.

04 ENTERPRISE AI TOP PICK

AI Company Brain

DIFFICULTY

STARTUP

An enterprise knowledge platform that turns scattered company data into a queryable knowledge graph with RAG search, workflow automation and collaborating multi-agent assistants.

CONTEXT

THE PROBLEM

Employees lose ~20% of their week searching for information across drives, wikis, tickets and chat. Knowledge walks out the door when people leave, and answers are inconsistent across teams.

EXISTING SOLUTIONS & GAPS

Keyword search returns documents, not answers. Existing RAG bots are single-shot and cannot reason across sources, trigger actions, or maintain an organisational memory graph.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Unified connector layer** (drive, wiki, tickets, chat)
- **Knowledge graph** + RAG answer engine with citations
- **Multi-agent assistant that** can act (create tickets, draft docs)
- **Admin console for** permissions and data governance

NON-FUNCTIONAL REQUIREMENTS

- **Respect source-level access** control (no data leaks)
- **Sub-2s answer latency** for common queries

MARKET SIZE

TAM: \$30B enterprise knowledge & search. **SAM: \$5.5B** India + SEA. **SOM: \$210M** mid-enterprise deployments in 3 years.

FUNCTIONAL REQUIREMENTS

- **Ingest and chunk** documents from multiple sources
- **Build entity/relationship** graph across content
- **Answer questions** with source citations
- **Agents that plan** and execute multi-step tasks

- **Auditable agent** actions
- **Scale to** millions of documents

ARCHITECTURE

SYSTEM ARCHITECTURE

Chat/search UI → API Gateway → AI Services (RAG, agents, graph) → Domain Microservices (connectors, permissions, actions) → PostgreSQL + Graph DB + Vector DB → Analytics Dashboard.

AI ARCHITECTURE

Hybrid retrieval (BM25 + dense) with reranking; graph-augmented RAG; multi-agent orchestration (planner + tool-executors) with MCP-style tool calling.

DATABASE DESIGN

Vector DB for embeddings, Graph DB (Neo4j) for entities, PostgreSQL for metadata and ACLs.

CLOUD ARCHITECTURE

Kubernetes; per-tenant vector namespaces; encrypted connector credentials in a vault; scheduled re-indexing jobs.

API FLOW

Query → permission filter → retrieval → rerank → LLM synthesis → cited answer. Action request → agent planner → tool calls → confirmation.

RECOMMENDED TECH STACK

Frontend: React. **Backend:** FastAPI/Node. **AI:** LangGraph/LlamaIndex, rerankers. **Data:** Qdrant/pgvector, Neo4j, PostgreSQL. **Infra:** Kubernetes, Redis.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Answer accuracy** with correct citations
- **Access-control correctness** (no unauthorised leakage)
- **Agent task-completion** rate
- **Retrieval quality** on enterprise queries

DEPLOYMENT GUIDE

Namespace-isolated tenants; blue-green; connector sandboxing; incremental re-index pipelines.

REFERENCES

Model Context Protocol, RAG evaluation frameworks (RAGAS), Neo4j graph patterns.

BONUS FEATURES

- **Slack/Teams native** app
- **Auto-generated onboarding** guides per role
- **Knowledge-gap detection** analytics

FUTURE SCOPE

Proactive briefings and org-wide decision memory.

Water-Energy Nexus Optimization

An AI optimisation platform that jointly manages desalination, purification and energy consumption to minimise cost and carbon while guaranteeing water supply.

CONTEXT

THE PROBLEM

Water treatment is energy-intensive and energy generation is water-intensive, yet the two are optimised separately. Utilities overspend on power and emit avoidable carbon while risking supply shortfalls.

EXISTING SOLUTIONS & GAPS

SCADA systems monitor but don't optimise across the water–energy boundary. No accessible platform forecasts demand and schedules treatment against dynamic energy prices and carbon intensity.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Demand-forecasting and** supply-planning dashboard
- **Joint optimisation engine** (treatment schedule vs. energy price)
- **Carbon and cost** analytics with scenario simulation
- **Alerting for** quality/supply risk thresholds

NON-FUNCTIONAL REQUIREMENTS

- **Optimisation runs within** operational time windows
- **Fail-safe defaults if** AI is unavailable

ARCHITECTURE

SYSTEM ARCHITECTURE

Operator dashboard → API Gateway → AI Services (forecast, optimiser) → Domain Microservices (assets, schedules, quality) → PostgreSQL + Timeseries + Vector DB → Analytics Dashboard.

AI ARCHITECTURE

Time-series forecasting (Prophet/temporal transformers) for demand and price; constrained optimisation (MILP / RL) for scheduling; anomaly detection on quality sensors.

DATABASE DESIGN

TimescaleDB for telemetry, PostgreSQL for assets/schedules, object storage for scenario snapshots.

RECOMMENDED TECH STACK

Frontend: React. **Backend:** FastAPI. **AI:** PyTorch, OR-Tools/Pyomo. **Data:** TimescaleDB, PostgreSQL. **Infra:** Kubernetes, Airflow.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Forecast accuracy (MAPE)** for demand and price
- **Cost/carbon reduction** vs. baseline schedule
- **Constraint satisfaction** (no supply violations)
- **Usability of** scenario simulation

DEPLOYMENT GUIDE

Advisory mode first (human dispatch), then supervised auto-dispatch; rollback to rule-based scheduler on anomaly.

REFERENCES

Grid carbon-intensity APIs, OR-Tools docs, WHO water-quality standards.

MARKET SIZE

TAM: \$14B water-tech + energy optimisation. **SAM: \$2.4B** India utilities. **SOM: \$110M** municipal/industrial deployments in 3 years.

FUNCTIONAL REQUIREMENTS

- **Forecast water demand** and energy price/carbon intensity
- **Optimise pump/treatment** schedules under constraints
- **Simulate what-if** scenarios
- **Track quality** parameters against limits

- **Auditable optimisation** decisions
- **Integrate with existing** SCADA read feeds

CLOUD ARCHITECTURE

Kubernetes; batch optimisation workers; timeseries ingestion pipeline; scenario compute on demand.

API FLOW

Telemetry + price feed → forecast → optimiser → schedule proposal → operator approval → dispatch. Quality breach → alert.

BONUS FEATURES

- **Leak-detection from** flow anomalies
- **Renewable-integration co-scheduling**
- **Regulatory ESG** reporting export

FUTURE SCOPE

City-scale multi-utility coordination and carbon-market participation.

Healthcare Without Hospitals

A home-first care ecosystem combining remote patient monitoring, telemedicine, AI triage and at-home diagnostics to deliver clinical-grade care outside hospital walls.

CONTEXT

THE PROBLEM

Hospitals are overloaded, rural access is poor, and chronic patients deteriorate between visits with no continuous monitoring. Triage bottlenecks delay care for those who need it most.

EXISTING SOLUTIONS & GAPS

Telemedicine apps offer video calls but lack continuous monitoring, safe AI triage with escalation, and integration of home diagnostic devices into a single clinical record.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Patient app** with vitals streaming from wearables/devices
- **AI symptom triage** with clinician escalation
- **Telemedicine consult** module with e-prescription
- **Clinician dashboard** with risk-stratified patient list

NON-FUNCTIONAL REQUIREMENTS

- **Clinical-grade reliability** and data privacy
- **Conservative triage** (bias toward escalation)

ARCHITECTURE

SYSTEM ARCHITECTURE

Patient/clinician apps → API Gateway → AI Services (triage, risk) → Domain Microservices (patients, vitals, consults, Rx) → PostgreSQL + Timeseries + Vector DB → Analytics Dashboard.

AI ARCHITECTURE

Triage classifier trained on symptom–urgency data with strong safety guardrails; anomaly detection on vitals; RAG over clinical guidelines for clinician support (assistive, not autonomous).

DATABASE DESIGN

PostgreSQL for records, Timeseries for vitals, Vector DB for guideline retrieval, encrypted object store for reports.

RECOMMENDED TECH STACK

Frontend: Flutter/React. **Backend:** FastAPI/Node. **AI:** PyTorch, guardrails. **Data:** PostgreSQL, TimescaleDB, pgvector. **Infra:** Kubernetes, WebRTC.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Triage safety: minimal** under-triage of urgent cases
- **Vitals-alert precision/recall**
- **End-to-end consult** experience
- **Privacy and** consent handling

DEPLOYMENT GUIDE

Clinician-supervised rollout; triage always escalatable to a human; staged device onboarding.

REFERENCES

FHIR/HL7 standards, WHO triage guidelines, wearable device SDKs.

MARKET SIZE

TAM: \$45B remote care & telehealth. **SAM: \$6.8B** India digital health. **SOM: \$260M** chronic-care & rural deployments in 3 years.

FUNCTIONAL REQUIREMENTS

- **Ingest vitals from** wearables and home devices
- **Triage engine mapping** symptoms to urgency
- **Video consult with** notes and prescription
- **Deterioration alerts** to care team

- **Consent-driven data** sharing
- **Audit trail for** every clinical action

CLOUD ARCHITECTURE

Kubernetes with encrypted PHI storage; strict tenancy; secure media for video; consent service.

API FLOW

Symptom input → triage → urgency + advice → (escalate → consult).
Vitals stream → anomaly → alert → clinician review.

BONUS FEATURES

- **Medication-adherence tracking**
- **Family-caregiver companion** view
- **Offline rural** kiosk mode

FUTURE SCOPE

Population health analytics and insurer-integrated value-based care.

An adaptive AI tutor that builds a live skills graph for each learner, personalises content, and tracks employability against real market demand.

CONTEXT

THE PROBLEM

One-size-fits-all courses ignore each learner's gaps; students finish programmes without knowing if they're job-ready. Educators lack visibility into who is falling behind and why.

EXISTING SOLUTIONS & GAPS

Most platforms deliver static video and quizzes with no true adaptivity, no skills-graph modelling, and no link between what is learned and what employers demand.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Adaptive tutor that** adjusts difficulty per learner
- **Per-learner skills graph** with mastery levels
- **Employability tracker mapping** skills to job roles
- **Educator analytics** dashboard

NON-FUNCTIONAL REQUIREMENTS

- **Fair, bias-aware** assessment
- **Responsive tutoring** latency

MARKET SIZE

TAM: \$28B global EdTech + upskilling. **SAM: \$4.9B** India. **SOM: \$170M** campus & upskilling deployments in 3 years.

FUNCTIONAL REQUIREMENTS

- **Diagnostic assessment to** seed the skills graph
- **Content recommendation** targeting weakest skills
- **Conversational tutoring with** hints and feedback
- **Job-role gap analysis** and learning paths

- **Scalable to** large cohorts
- **Transparent mastery** scoring

ARCHITECTURE

SYSTEM ARCHITECTURE

Learner/educator UI → API Gateway → AI Services (tutor, assessment, recommender) → Domain Microservices (skills graph, content, jobs) → PostgreSQL + Graph DB + Vector DB → Analytics Dashboard.

AI ARCHITECTURE

Knowledge-tracing model (DKT/BKT) for mastery; RAG tutor with Socratic prompting; embedding match between skills and job-description corpus.

DATABASE DESIGN

Graph DB for skills, PostgreSQL for users/progress, Vector DB for content and job matching.

RECOMMENDED TECH STACK

Frontend: React. **Backend:** FastAPI/Node. **AI:** PyTorch, LangChain. **Data:** Neo4j, PostgreSQL, pgvector. **Infra:** Kubernetes.

CLOUD ARCHITECTURE

Kubernetes; content CDN; batch analytics jobs; embeddings refresh pipeline.

API FLOW

Attempt → knowledge-tracing update → next-best content. Skills graph → job corpus match → gap report + path.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Learning-gain improvement** vs. static baseline
- **Accuracy of** mastery/knowledge tracing
- **Relevance of** job-role gap analysis
- **Educator dashboard** usefulness

DEPLOYMENT GUIDE

Cohort pilots; A/B test tutoring strategies; gradual feature rollout.

REFERENCES

*Knowledge-tracing literature, O*NET/skills taxonomies, learning-analytics standards (xAPI).*

BONUS FEATURES

- **Peer-cohort collaboration** features
- **Micro-credential/certificate issuance**
- **Employer-facing talent** discovery view

FUTURE SCOPE

Lifelong learning passport portable across institutions and employers.

Fragmented Manufacturing Cloud

A shared-capacity network where SMEs list idle manufacturing capacity and an AI engine schedules jobs, matches suppliers and orchestrates distributed production.

CONTEXT

THE PROBLEM

Small manufacturers run below capacity while buyers struggle to find reliable production slots. Matching is manual, trust is low, and scheduling across many small shops is chaotic.

EXISTING SOLUTIONS & GAPS

B2B marketplaces list vendors but don't schedule capacity in real time, verify quality, or optimise multi-shop job splitting — leaving idle machines and unmet demand.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Capacity marketplace** for SMEs and buyers
- **AI scheduling engine** that splits/assigns jobs
- **Supplier matching with** capability & trust scoring
- **Order-tracking and** settlement dashboard

NON-FUNCTIONAL REQUIREMENTS

- **Fair, explainable** matching
- **Scale to** thousands of shops

ARCHITECTURE

SYSTEM ARCHITECTURE

Marketplace UI → API Gateway → AI Services (matching, scheduling) → Domain Microservices (listings, orders, trust) → PostgreSQL + Vector DB → Analytics Dashboard.

AI ARCHITECTURE

Constraint-based scheduler (OR-Tools) plus ML capability-matching from embeddings; trust score from delivery history and reviews.

DATABASE DESIGN

PostgreSQL for listings/orders, Vector DB for capability matching, ledger table for settlements.

RECOMMENDED TECH STACK

Frontend: React. **Backend:** Node/FastAPI. **AI:** OR-Tools, embeddings. **Data:** PostgreSQL, Qdrant. **Infra:** Kubernetes, Kafka.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Quality of matching** (utilisation + on-time)
- **Scheduling optimality** vs. baseline
- **Marketplace UX** for both sides
- **Trust-score robustness**

DEPLOYMENT GUIDE

Region-by-region launch; onboarding verification; phased trust features.

REFERENCES

OR-Tools scheduling, MSME registries, B2B escrow patterns.

MARKET SIZE

TAM: \$18B distributed manufacturing. **SAM: \$3.6B** India MSME manufacturing. **SOM: \$150M** capacity-sharing network in 3 years.

FUNCTIONAL REQUIREMENTS

- **List machine capability**, capacity and calendar
- **Match a job** to optimal shop(s)
- **Schedule across constraints** (deadline, cost, location)
- **Track order** milestones to delivery
- **Reliable status** accuracy
- **Dispute-resistant records**

CLOUD ARCHITECTURE

Kubernetes; event-driven order state machine; notification service; escrow/settlement integration.

API FLOW

Job posted → match candidates → optimise assignment → confirm → milestone tracking → settle.

BONUS FEATURES

- **Logistics/shipping integration**
- **Quality-inspection proof** uploads
- **Working-capital financing** hooks

FUTURE SCOPE

National distributed-manufacturing grid with dynamic pricing.

Agent-to-Agent Payment Protocol

A secure protocol and platform enabling autonomous AI agents to transact with each other under policy controls, spending limits and a complete audit ledger.

CONTEXT

THE PROBLEM

As AI agents begin to buy compute, data and services autonomously, there is no safe standard for agent-to-agent payments — creating risks of runaway spend, fraud and zero accountability.

EXISTING SOLUTIONS & GAPS

Human payment rails assume a human approver. There is no policy engine, identity model or auditable settlement layer designed for machine-initiated transactions.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Agent identity** & wallet abstraction
- **Policy engine** (limits, allowlists, approvals)
- **Transaction ledger** with full audit
- **Monitoring console** for spend and anomalies

NON-FUNCTIONAL REQUIREMENTS

- **Strong authn/authz** for agents
- **Tamper-evident audit** ledger

MARKET SIZE

TAM: \$9B emerging agentic-commerce infrastructure. **SAM: \$1.4B** early adopters. **SOM: \$70M** protocol/platform in 3 years.

FUNCTIONAL REQUIREMENTS

- **Issue verifiable** agent identities
- **Enforce per-agent** spending policies
- **Execute and** settle transactions
- **Detect and** block anomalous spending

- **Deterministic policy** enforcement
- **Low-latency authorisation**

ARCHITECTURE

SYSTEM ARCHITECTURE

Agent SDK → API Gateway → AI Services (fraud/anomaly) → Domain Microservices (identity, policy, ledger, settlement) → PostgreSQL + Vector DB → Analytics Dashboard.

AI ARCHITECTURE

Anomaly detection on transaction patterns; risk scoring per transaction; optional LLM policy-intent parsing with strict deterministic enforcement.

DATABASE DESIGN

PostgreSQL for identities/policies, append-only ledger (or L2 chain) for settlement, Vector DB for pattern search.

RECOMMENDED TECH STACK

Frontend: React console. **Backend:** Go/Rust core, FastAPI services. **AI:** PyTorch. **Data:** PostgreSQL, ledger. **Infra:** Kubernetes, Vault.

CLOUD ARCHITECTURE

Kubernetes; HSM/key management; append-only ledger; optional blockchain settlement layer.

API FLOW

Agent initiates → identity verify → policy check → risk score → approve/deny → settle → ledger + audit.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Policy-enforcement correctness** (no bypass)
- **Fraud/anomaly detection** quality
- **Audit completeness** and tamper-evidence
- **Authorisation latency**

DEPLOYMENT GUIDE

Sandbox with test funds first; capped live limits; kill-switch for runaway agents.

REFERENCES

OAuth/mTLS, verifiable credentials (W3C), payment-tokenisation standards.

BONUS FEATURES

- **Multi-currency /** stablecoin support
- **Delegated human-approval** escalation
- **Reputation system** for agents

FUTURE SCOPE

Open interoperable standard across agent ecosystems.

AI Hospital Administration OS

An AI back-office platform that automates hospital administration — scheduling, billing, claims, records and resource allocation — freeing staff to focus on patients.

CONTEXT

THE PROBLEM

Hospitals lose 20–30% of staff time to paperwork: appointment juggling, insurance claims, coding errors and bed management.

Administrative overload drives burnout and billing losses.

EXISTING SOLUTIONS & GAPS

Legacy HIS software is rigid and manual. There is no AI layer that automates claims, predicts resource needs, and reduces coding/billing errors end to end.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Smart scheduling** for staff, beds and OTs
- **AI-assisted medical coding** and claims automation
- **Records digitisation** and summarisation
- **Operations analytics** dashboard

NON-FUNCTIONAL REQUIREMENTS

- **Accurate, auditable** financial actions
- **PHI privacy** and access control

ARCHITECTURE

SYSTEM ARCHITECTURE

Admin UI → API Gateway → AI Services (coding, forecasting) → Domain Microservices (scheduling, billing, records) → PostgreSQL + Vector DB → Analytics Dashboard.

AI ARCHITECTURE

NLP for note-to-code mapping; demand forecasting for resources; claim-denial risk prediction; RAG over payer rules.

DATABASE DESIGN

PostgreSQL for operations/billing, Vector DB for payer-rule retrieval, encrypted store for records.

RECOMMENDED TECH STACK

Frontend: React. **Backend:** FastAPI/Node. **AI:** PyTorch, NLP. **Data:** PostgreSQL, pgvector. **Infra:** Kubernetes.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Coding/billing accuracy** vs. manual
- **Claim-denial reduction**
- **Scheduling efficiency** gains
- **Privacy compliance**

DEPLOYMENT GUIDE

Human approval on financial actions; module-by-module rollout; reconciliation checks.

REFERENCES

ICD/CPT coding, FHIR, national claims/insurance frameworks.

MARKET SIZE

TAM: \$22B healthcare admin automation. **SAM:** \$3.9B India hospital operations. **SOM:** \$160M mid-hospital deployments in 3 years.

FUNCTIONAL REQUIREMENTS

- **Optimise appointment** and resource schedules
- **Auto-suggest billing** codes from notes
- **Validate and** submit insurance claims
- **Forecast bed/staff** demand

- **High availability** during operating hours
- **Explainable coding** suggestions

CLOUD ARCHITECTURE

Kubernetes; encrypted PHI storage; HIS/EMR integration adapters; job scheduler.

API FLOW

Note → coding suggestion → human confirm → claim build → validate → submit → track denial. Demand data → forecast → schedule.

BONUS FEATURES

- **Patient no-show** prediction
- **Inventory/pharmacy automation**
- **Revenue-cycle analytics**

FUTURE SCOPE

Fully integrated revenue-cycle and value-based-care reporting.

Blockchain Usability Layer

A usability layer that hides blockchain complexity — wallet abstraction, gasless onboarding and simple identity — so mainstream users can adopt Web3 without friction.

CONTEXT

THE PROBLEM

Seed phrases, gas fees and confusing wallets block mainstream adoption. Onboarding drop-off exceeds 90% for non-crypto users, and identity/recovery is fragile.

EXISTING SOLUTIONS & GAPS

Current wallets expose raw crypto primitives. There is no clean abstraction giving email-style login, sponsored gas and social recovery in one embeddable layer.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Embeddable smart-wallet** SDK
- **Gasless (sponsored)** transaction relayer
- **Email/social login** with account abstraction
- **Social-recovery** and identity module

NON-FUNCTIONAL REQUIREMENTS

- **Non-custodial security** guarantees
- **Fast transaction** UX

MARKET SIZE

TAM: \$11B Web3 infrastructure. **SAM: \$1.8B** onboarding/wallet tooling. **SOM: \$85M** consumer-app integrations in 3 years.

FUNCTIONAL REQUIREMENTS

- **Create wallets** without seed phrases
- **Sponsor gas** for user actions
- **Sign transactions** via familiar login
- **Recover accounts** via guardians

- **Auditable relayer** spend
- **Cross-chain** readiness

ARCHITECTURE

SYSTEM ARCHITECTURE

App SDK → API Gateway → AI Services (fraud/abuse detection) → Domain Microservices (wallet, relayer, identity) → PostgreSQL + Vector DB → Analytics Dashboard. On-chain: account-abstraction contracts.

AI ARCHITECTURE

Abuse/sybil detection on relayer usage; risk scoring for sponsored transactions.

DATABASE DESIGN

PostgreSQL for accounts/policies, Vector DB for abuse-pattern search, chain indexer store.

RECOMMENDED TECH STACK

Frontend: React SDK. **Backend:** Node/Go. **Chain:** ERC-4337 account abstraction, Solidity. **Data:** PostgreSQL. **Infra:** Kubernetes.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Onboarding conversion** improvement
- **Transaction success rate** & UX latency
- **Security** of key management/recovery
- **Relayer abuse** resistance

DEPLOYMENT GUIDE

Testnet first; capped relayer budgets; progressive mainnet rollout.

REFERENCES

ERC-4337 account abstraction, MPC wallet patterns, W3C DID.

CLOUD ARCHITECTURE

Kubernetes; key management/MPC; bundler/relayer infra; RPC node access.

API FLOW

Login → derive smart account → user action → relayer sponsors gas → bundle → on-chain → status. Recovery → guardian approval → key rotate.

BONUS FEATURES

- **Multi-chain** support
- **Fiat on-ramp** integration
- **Portable reputation/identity**

FUTURE SCOPE

Universal Web3 login standard embedded across consumer apps.

Global Compliance Passport

A portable compliance profile that lets startups reuse verified KYC, legal and regulatory credentials across countries, banks, investors and platforms.

CONTEXT

THE PROBLEM

Every new market, bank or investor demands the same compliance documents from scratch. Startups waste months and heavy cost re-proving identity and legitimacy across fragmented jurisdictions.

EXISTING SOLUTIONS & GAPS

Compliance data is siloed per institution and country. No portable, verifiable, reusable profile exists that institutions can trust without re-collecting everything.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Verifiable compliance profile** (KYC/KYB, legal docs)
- **Credential-verification and** sharing engine
- **Jurisdiction rule-mapping** module
- **Institution-facing verification** portal

NON-FUNCTIONAL REQUIREMENTS

- **Tamper-proof, verifiable** credentials
- **Consent-driven, privacy-preserving** sharing

ARCHITECTURE

SYSTEM ARCHITECTURE

Startup/institution UI → API Gateway → AI Services (doc extraction, rule mapping) → Domain Microservices (profile, credentials, consent) → PostgreSQL + Vector DB → Analytics Dashboard.
Verifiable-credential layer for trust.

AI ARCHITECTURE

Document understanding (OCR + IDP) to extract entities; RAG over jurisdiction rules to map requirements; risk/fraud checks on documents.

DATABASE DESIGN

PostgreSQL for profiles/consent, Vector DB for rule retrieval, credential registry.

RECOMMENDED TECH STACK

Frontend: React. **Backend:** FastAPI/Node. **AI:** OCR/IDP, RAG. **Data:** PostgreSQL, pgvector, VC registry. **Infra:** Kubernetes, Vault.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Accuracy** of document extraction/verification
- **Correctness** of jurisdiction rule mapping
- **Privacy &** consent enforcement
- **Institution trust/verification** UX

DEPLOYMENT GUIDE

Start with a few jurisdictions; expand rule packs; audited credential issuance.

REFERENCES

W3C Verifiable Credentials, FATF KYC/AML guidelines, eIDAS.

MARKET SIZE

TAM: \$20B RegTech/compliance. **SAM: \$3.1B** cross-border startup compliance. **SOM: \$130M** passport-network in 3 years.

FUNCTIONAL REQUIREMENTS

- **Collect and verify** compliance documents once
- **Issue verifiable** credentials
- **Map requirements** per country/institution
- **Share selectively** with consent
- **Auditable access** history
- **Regulatory-grade data** handling

CLOUD ARCHITECTURE

Kubernetes; encrypted document vault; verifiable-credential issuance; consent ledger.

API FLOW

Upload docs → extract/verify → issue credentials → institution requests → consent → selective disclosure → audit.

BONUS FEATURES

- **Selective zero-knowledge** disclosure
- **Continuous-compliance monitoring**
- **Investor data-room** integration

FUTURE SCOPE

Global standard recognised by regulators and banks.

AI Disaster Response Platform

A multi-agent emergency-coordination platform that fuses real-time data, allocates resources and coordinates responders during disasters.

CONTEXT

THE PROBLEM

In disasters, information is fragmented across agencies, resource allocation is manual and slow, and coordination failures cost lives in the critical first hours.

EXISTING SOLUTIONS & GAPS

Existing systems are single-agency dashboards. None fuse multi-source signals, run AI resource-allocation, and coordinate multiple responder agents in real time.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Real-time situational-awareness** map
- **Multi-agent resource-allocation** engine
- **Responder coordination** & tasking console
- **Public alerting** and reporting channel

NON-FUNCTIONAL REQUIREMENTS

- **Extreme reliability** under load/failure
- **Low-latency updates**

MARKET SIZE

TAM: \$13B disaster-management tech. **SAM: \$2.0B** India + emerging markets. **SOM: \$100M** agency deployments in 3 years.

FUNCTIONAL REQUIREMENTS

- **Fuse sensor, social** and report data
- **Predict impact** zones and needs
- **Allocate/route resources** optimally
- **Coordinate responder** tasks and status

- **Graceful degradation** offline
- **Auditable decisions**

ARCHITECTURE

SYSTEM ARCHITECTURE

Command UI → API Gateway → AI Services (impact prediction, allocation, multi-agent) → Domain Microservices (incidents, resources, responders) → PostgreSQL + PostGIS + Vector DB → Analytics Dashboard.

AI ARCHITECTURE

Impact-prediction models; optimisation for resource routing; multi-agent orchestration for parallel tasking; NLP for triaging citizen reports.

DATABASE DESIGN

PostgreSQL + PostGIS for geospatial incidents, Timeseries for feeds, Vector DB for report clustering.

RECOMMENDED TECH STACK

Frontend: React + maps. **Backend:** Go/FastAPI. **AI:** PyTorch, OR-Tools. **Data:** PostgreSQL+PostGIS. **Infra:** Kubernetes multi-region.

CLOUD ARCHITECTURE

Kubernetes with multi-region failover; offline-capable edge sync; resilient messaging.

API FLOW

Signals → fusion → impact prediction → allocation → responder tasking → status → public alerts.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Quality of** resource-allocation decisions
- **Timeliness of** situational updates
- **Resilience under** simulated failure
- **Coordination usability** for responders

DEPLOYMENT GUIDE

Drill-tested rollout; multi-region redundancy; offline fallback packages.

REFERENCES

Sphere standards, OpenStreetMap/HOT, incident-command frameworks.

BONUS FEATURES

- **Drone/satellite damage** assessment
- **Volunteer crowd-coordination**
- **Post-event recovery** analytics

FUTURE SCOPE

National early-warning and cross-border disaster coordination.

Circular Waste Intelligence Platform

An AI platform for automated waste sorting, a recycling marketplace and ESG analytics that turns waste streams into a circular-economy resource.

CONTEXT

THE PROBLEM

Most recyclable material is lost to poor sorting; recyclers can't find consistent feedstock; and companies struggle to measure and report ESG/waste metrics credibly.

EXISTING SOLUTIONS & GAPS

Sorting is manual and error-prone, recycling supply chains are informal and untraceable, and ESG reporting lacks verifiable waste data.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Vision-based waste-sorting** classifier
- **Recycling marketplace** (waste supply ↔ recyclers)
- **ESG & diversion** analytics dashboard
- **Traceability records** for material flow

NON-FUNCTIONAL REQUIREMENTS

- **Robust classification** across conditions
- **Trustworthy traceability** records

MARKET SIZE

TAM: \$17B circular-economy & waste-tech. **SAM: \$2.7B** India waste management. **SOM: \$120M** sorting+marketplace in 3 years.

FUNCTIONAL REQUIREMENTS

- **Classify waste by** type from images
- **Match waste supply** to recycler demand
- **Track diversion** and material flow
- **Generate ESG** reports

- **Scalable marketplace**
- **Verifiable ESG** metrics

ARCHITECTURE

SYSTEM ARCHITECTURE

Web/edge UI → API Gateway → AI Services (vision, matching) → Domain Microservices (listings, traceability, ESG) → PostgreSQL + Vector DB → Analytics Dashboard. Edge cameras for on-belt sorting.

AI ARCHITECTURE

Object detection/classification for material types; embedding-based supply–demand matching; forecasting of waste volumes.

DATABASE DESIGN

PostgreSQL for listings/traceability, Vector DB for matching, object store for evidence images.

RECOMMENDED TECH STACK

Frontend: React. **Backend:** Node/FastAPI. **AI:** PyTorch, edge runtime. **Data:** PostgreSQL, Qdrant. **Infra:** Kubernetes, MQTT.

CLOUD ARCHITECTURE

Kubernetes with edge inference; object storage for imagery; marketplace event bus.

API FLOW

Image → classify → route to bin/listing → match recycler → transaction → traceability + ESG update.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Sorting classification** accuracy
- **Marketplace matching** quality
- **Traceability integrity**
- **Credibility of** ESG reporting

DEPLOYMENT GUIDE

Pilot at one facility; expand edge units; phased marketplace launch.

REFERENCES

GRI/ESG reporting standards, waste-classification datasets (TrashNet), EPR regulations.

BONUS FEATURES

- **Reward/incentive system** for recyclers
- **Carbon-credit estimation**
- **Municipal integration**

FUTURE SCOPE

City-scale circular-economy operating system.

Autonomous Startup Operating System

An AI 'CEO assistant' for founders that coordinates finance, hiring, legal, go-to-market and fundraising through specialised collaborating agents.

CONTEXT

THE PROBLEM

Early founders juggle every function with no team, drowning in operational work across finance, legal, hiring and GTM. Critical tasks slip and decisions are made without expertise.

EXISTING SOLUTIONS & GAPS

Point tools (accounting, ATS, CRM) don't talk to each other and none provide an orchestrating layer that reasons across functions and executes tasks for a founder.

WHAT TO BUILD

EXPECTED DELIVERABLES

- **Multi-agent orchestrator** across business functions
- **Finance agent** (bookkeeping, runway, reports)
- **Hiring, legal** and GTM agents
- **Founder command** dashboard

NON-FUNCTIONAL REQUIREMENTS

- **Human approval** on high-stakes actions
- **Auditable agent** decisions

ARCHITECTURE

SYSTEM ARCHITECTURE

Founder UI → API Gateway → AI Services (orchestrator + function agents) → Domain Microservices (finance, hiring, legal, GTM, integrations) → PostgreSQL + Vector DB → Analytics Dashboard.

AI ARCHITECTURE

Planner-executor multi-agent system with tool calling (MCP); RAG over templates and playbooks; per-domain specialised prompts and guardrails.

DATABASE DESIGN

PostgreSQL for tasks/entities, Vector DB for playbooks/templates, integration token vault.

RECOMMENDED TECH STACK

Frontend: React. **Backend:** FastAPI/Node. **AI:** LangGraph/multi-agent, MCP tools. **Data:** PostgreSQL, pgvector. **Infra:** Kubernetes, Vault.

HOW YOU'RE JUDGED

EVALUATION CRITERIA

- **Task-completion quality** across functions
- **Safety** of high-stakes action gating
- **Integration reliability**
- **Real founder** time saved

DEPLOYMENT GUIDE

Advisory mode first; approval gates on money/legal; staged agent activation.

REFERENCES

Model Context Protocol, multi-agent frameworks, startup operations playbooks.

MARKET SIZE

TAM: \$19B founder/SMB operations tooling. **SAM: \$3.0B** startup operations. **SOM: \$140M** autonomous-ops platform in 3 years.

FUNCTIONAL REQUIREMENTS

- **Route founder goals** to specialised agents
- **Draft finance/legal/GTM** artefacts
- **Track tasks and** deadlines across functions
- **Surface decisions** needing founder input

- **Secure handling** of financial/legal data
- **Reliable tool** integrations

CLOUD ARCHITECTURE

Kubernetes; secure credential vault for integrations; task/state store; approval workflow service.

API FLOW

Founder goal → orchestrator plan → dispatch to agents → tool actions → approval gate → execute → report.

BONUS FEATURES

- **Fundraising CRM** & investor updates
- **Automated financial** forecasting
- **Board-report** generation

FUTURE SCOPE

Near-autonomous operations for micro-startups with founder oversight.

Build something worth deploying

Pick the problem that excites your team most — not the easiest one. Judges reward a **focused, working, well-architected slice** over an unfinished everything. Scope tight, ship something real, and tell a clear story in your pitch.

Judging snapshot.

- **Working, deployed** build
- **Sound architecture** & sensible AI use
- **Real problem–solution** fit
- **Security & scalability** thinking
- **Clarity** of the final pitch
- **Bonus features** & polish



Form your team and get ready for 36 hours of innovation.

VYNEDAM · DRIVEN BY VISION, DESIGNED BY DATA